# BEYOND PIANO: CROSS-INSTRUMENT MIDI VELOCITY ESTIMATION VIA DIFFERENTIABLE SOUNDFONT PROXIES

**Zhanhong He**[1] **Hanyu Meng**[2] **David (Defeng) Huang**[1] **Roberto Togneri** [1]
[1] University of Western Australia, Perth, Australia
[2] University of New South Wales, Sydney, Australia
zhanh.he.uwa@gmail.com, hanyu.meng@unsw.edu.au, {david.huang; roberto.togneri}@uwa.edu.au

## ABSTRACT

Many music datasets contain MIDI notes but lack reliable velocities, defaulting to a constant value. This absence is especially problematic outside the piano domain, as velocity is a core component for expressive rendering, music generation, and performance analysis. This paper studies cross-instrument MIDI velocity estimation in this label-scarce setting. Starting from a piano-trained velocity estimator, we recast target-instrument adaptation as predicting renderer-conditioned velocities whose rendering matches the dynamics of the performance audio. This adaptation can be driven by either differentiable synthesizers (Diff-Synth) or our proposed differentiable SoundFont proxies (Diff-SFProxy). We highlight the Diff-SFProxy: it supervises velocity through note-wise, loudness-related acoustic parameters rather than waveform reconstruction, focusing gradients on velocity-dependent behavior. Experiments on piano and guitar show that Diff-SFProxy is effective for cross-instrument MIDI velocity estimation, while waveform-domain Diff-Synth degrades performance.

## 1. INTRODUCTION

MIDI velocity is the per-note control parameter in symbolic music systems, controlling expressive aspects of rendering such as amplitude and timbre. Since its effect varies across instruments and recording conditions, velocity is related to but cannot be directly replaced by loudness measurement [1, 2]. Reliable velocity labels are crucial for expressive rendering [3, 4], music generation [5, 6], performance analysis [7] and data curation [8]. However, most publicly available MIDI files lack velocity corresponding to human performance. A common practice in music software is to map score-level dynamic markings (such as $p$ and $f$) to fixed velocities, or default to 64 when unavailable [1]. Such velocities carry minimal expressive information. By contrast, learning-based velocity estimation from performance audio works well, but almost entirely in the piano domain [2, 9, 10, 11]. The reason is data: piano datasets such as MAESTRO [12] are enabled by sensor-equipped Yamaha Disklavier pianos, which provide note-aligned velocity labels from recorded performances. For guitar, violin, and other instruments, public datasets [13, 14, 15] provide audio and aligned notes but no velocity labels.

This paper studies label-scarce cross-instrument MIDI velocity estimation. In music transcription, a common strategy is to train on synthesized audio, where symbolic labels are available by construction, and transfer the model to real performances [16, 17, 18]. Recent works have used this strategy for synthetic velocity supervision [19, 20], but the synthetic-to-real gap remains. We instead take a more direct route. Starting from a velocity estimator (VeloEst) trained on piano, we adapt it to target instruments using real performance audio, predicting velocities whose rendering matches the dynamics of the recording.

For Diff-Synth, while several implementations exist for each instrument, we adopt DDSP-Piano [21] and DDSP-Guitar-Synth [22], both of which are MIDI-oriented. Velocity estimates are rendered into audio with aligned note events, and an audio-based loss against the real recording drives gradients back into VeloEst. Yet Diff-Synth only approximates the real instrument, so waveform or spectral objectives spend much of their gradient on residual timbral mismatch rather than on velocity-dependent behavior.

Our proposed Diff-SFProxy follows the neural-proxy strategy for black-box audio systems [23, 24, 25]. SoundFont is a widely adopted software-based instrument format for rendering MIDI into audio. Because this rendering is non-differentiable, it cannot pass gradients to VeloEst, which motivates a proxy. Diff-SFProxy is not asked to reproduce waveforms. Instead, it maps note events to loudness-related acoustic parameters: pitch-conditioned harmonic energy (PHE) and onset-window spectral flux (OSF). This design yields two benefits over Diff-Synth: (i) many SoundFonts can plug in off the shelf, whereas Diff-Synth requires a dedicated design per instrument; (ii) gradients focus on velocity-relevant intensity and attack cues rather than waveform details sensitive to recording and timbral mismatch. [1]

The contributions of this paper are threefold. First, we formulate label-scarce cross-instrument MIDI velocity estimation and investigate two differentiable adaptation

© Zhanhong He, Hanyu Meng, David (Defeng) Huang, Roberto Togneri. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Zhanhong He, Hanyu Meng, David (Defeng) Huang, Roberto Togneri, "Beyond Piano: Cross-Instrument MIDI Velocity Estimation via Differentiable SoundFont Proxies", in *Proc. of the 27th Int. Society for Music Information Retrieval Conf.,* Abu Dhabi, UAE, 2026.

[1] Code is available at https://github.com/zhanh-he/sfproxy-velocity-estimation

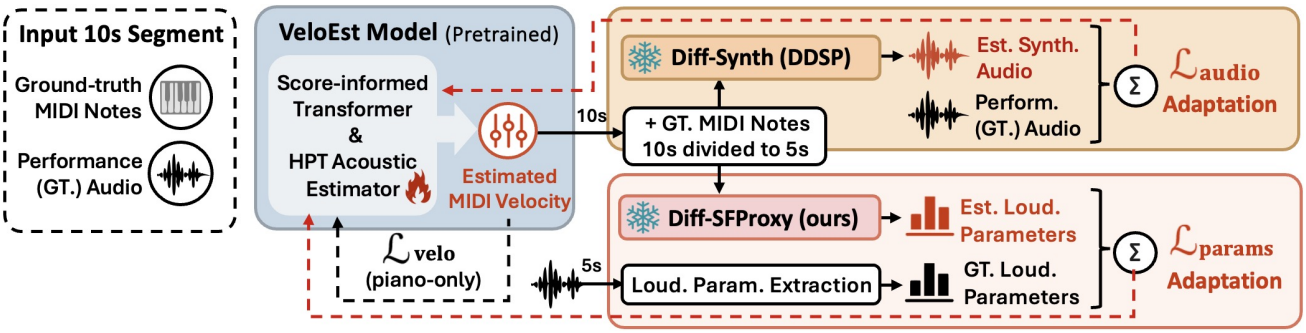



**Figure 1**. Overview of the cross-instrument velocity estimation framework. VeloEst is trained with ground-truth velocity labels on piano ($\mathcal{L}_{\text{velo}}$), then fine-tuned for target instruments through one of two adaptation strategies. Diff-Synth provides waveform-domain supervision ($\mathcal{L}_{\text{audio}}$). Diff-SFProxy provides note-wise loudness-parameter supervision ($\mathcal{L}_{\text{params}}$).

strategies: Diff-Synth and Diff-SFProxy. Second, we introduce Diff-SFProxy itself, a compact SoundFont proxy that supervises on note-wise loudness-related acoustic parameters rather than waveforms. Third, we validate the proposed framework on piano and guitar recordings. To evaluate without the use of ground-truth velocity labels, we adopt Bark-scale specific loudness (BSSL) and Bark-scale total loudness (BSTL) [26, 27] to assess whether predicted velocities reproduce the perceived loudness of the original recording, reflecting its performance dynamics.

## 2. CROSS-INSTRUMENT MIDI VELOCITY ESTIMATION FRAMEWORK

### 2.1 VeloEst: Pretrained on Piano

Figure 1 shows the overall framework. We adopt the score-HPT model of [11], which achieved state-of-the-art performance in MIDI velocity estimation, together with its publicly available checkpoint trained on MAESTRO using ground-truth velocity supervision $\mathcal{L}_{\text{velo}}$. Serving as the shared front-end, VeloEst takes log-Mel spectrograms computed at 100 fps from 22050 Hz mono audio (FFT size 2048, hop 221 samples, Hann window) and aligned note information to estimate MIDI velocities. During cross-instrument adaptation, only VeloEst is updated, while the Diff-Synth and Diff-SFProxy back-ends remain frozen.

### 2.2 VeloEst Adaptation via Diff-Synth

During the adaptation, only VeloEst parameters are updated. Diff-Synth is a frozen, instrument-specific differentiable synthesizer. Given note events $E$ (pitch, onset, duration) derived from the aligned score and predicted velocities $\hat{V}$, Diff-Synth renders audio $\tilde{a}$. The adaptation loss is the multi-scale spectral loss proposed in [28], which is widely adopted in the differentiable synthesizer family [21, 22, 29]. We apply it between the rendered audio $\tilde{a}$ and the real performance audio $a$,

$$\mathcal{L}_{\text{audio}} = \sum_{\omega \in \Omega} \Big( \big\| |\text{STFT}_\omega(\tilde{a})| - |\text{STFT}_\omega(a)| \big\|_1 + \big\| \log(|\text{STFT}_\omega(\tilde{a})| + \epsilon) - \log(|\text{STFT}_\omega(a)| + \epsilon) \big\|_1 \Big), \tag{1}$$

where $\Omega = \{2048, 1024, 512, 256, 128, 64\}$ as in [28]. For each $\omega \in \Omega$, $\text{STFT}_\omega$ uses an $\omega$-sample Hann window with 75% overlap and an $\omega$-point FFT. We apply $\log(|\cdot| + \epsilon)$ with $\epsilon = 10^{-7}$ for numerical stability.

### 2.3 VeloEst Adaptation via Diff-SFProxy

As in the Diff-Synth path, only VeloEst parameters are updated. Diff-SFProxy is a frozen differentiable proxy $P$ of an instrument-specific SoundFont. Let $F$ be the note-wise loudness-related acoustic parameter extractor, detailed in Section 3.1, that reads audio conditioned on the note events $E$. Given predicted velocities $\hat{v}$, the two feature streams are

$$Z = F(a, E), \qquad \tilde{Z} = P(E, \hat{v}). \tag{2}$$

The adaptation loss is the Huber (Smooth-$L_1$) distance between the proxy prediction $\tilde{Z}$ and the extracted loudness-related acoustic parameters $Z$,

$$\mathcal{L}_{\text{params}} = \text{Huber}_\beta\left(\tilde{Z} - Z\right), \tag{3}$$

with the default $\beta = 1.0$. With $Z_n = (\text{PHE}_n, \text{OSF}_n)$ defined in Section 3.1, the Huber loss averages over both components and all valid notes, weighting PHE and OSF equally (1:1). The Huber form tolerates outliers from onset contamination, such as overlapping notes, while the parameter-space objective concentrates gradients on note intensity and attack behaviour rather than on waveform details weakly related to velocity.

### 2.4 Shared Regularizers and Training Objective

Diff-Synth and Diff-SFProxy adaptation share two auxiliary regularizers on the predicted velocity vector $\hat{v} = (\hat{v}_1, \ldots, \hat{v}_N) \in [0, 1]^N$, where $N$ is the number of notes in the segment. The first is an anti-collapse regularizer,

$$\mathcal{L}_{\text{anti}} = (\bar{v} - \mu)^2 + \max\big(0,\ \sigma^2_{\text{min}} - \text{Var}(\hat{v})\big), \tag{4}$$

where $\bar{v}$ and $\text{Var}(\hat{v})$ are the empirical mean and variance of the predicted velocities, $\mu = 0.5$ is the desired operating point, and $\sigma^2_{\text{min}} = 0.01$ is the minimum-variance threshold. The mean term resists drift towards extremes, while

the hinge variance term penalises collapse towards a single output value. The second is a soft saturation penalty,

$$\mathcal{L}_{\text{satu}} = \frac{1}{N}\sum_{n=1}^{N}\big[\max(0,\, \hat{v}_n - \tau)\big]^2, \tag{5}$$

which quadratically penalises predictions above a near-maximum threshold $\tau = 0.95$, discouraging VeloEst from pushing predictions into the saturated region where audio-derived gradients become uninformative.

$$\mathcal{L} = \mathcal{L}_{\text{params}} + 0.2\,\mathcal{L}_{\text{anti}} + 0.4\,\mathcal{L}_{\text{satu}}. \tag{6}$$

where $\mathcal{L}_{\text{main}} \in \{\mathcal{L}_{\text{audio}}, \mathcal{L}_{\text{params}}\}$ selects the active setting. For both adaptation branches, the loss weights for $\mathcal{L}_{\text{main}}$, $\mathcal{L}_{\text{anti}}$, and $\mathcal{L}_{\text{satu}}$ are fixed at 1:0.2:0.4, based on grid search.

## 3. DIFFERENTIABLE SOUNDFONT PROXY

### 3.1 Loudness-Related Acoustic Parameters

To characterize the velocity-dependent acoustic behavior of a SoundFont, we use two note-wise loudness-related parameters adapted from established MIR feature families. The extractor $F$ computes these parameters from the magnitude STFT $X(k,t) = |\mathrm{STFT}(k,t)|$, using the same FFT settings as Section 2.1. Here $k$ indexes frequency bins and $t$ indexes STFT frames. For each valid note $n \in \{1,\dots,N\}$, the extractor stacks PHE and OSF into

$$Z_n = \big(\mathrm{PHE}_n,\ \mathrm{OSF}_n\big) \in \mathbb{R}^2, \tag{7}$$

and stacks all note-wise pairs into $Z = (Z_1,\dots,Z_N) \in \mathbb{R}^{N\times 2}$. Each parameter is defined below.

**Pitch-conditioned harmonic energy (PHE)** draws on harmonic-structure-based note-intensity estimation [30]. It accumulates magnitude energy around the retained harmonics of the note pitch:

$$\mathrm{PHE}_n = \log\left(1 + \sum_{h=1}^{H_n} \frac{1}{|\mathcal{B}_{n,h}|\,|\mathcal{T}_n|} \sum_{k\in\mathcal{B}_{n,h}} \sum_{t\in\mathcal{T}_n} X(k,t)^2\right) \tag{8}$$

where $h$ indexes harmonics, $\mathcal{B}_{n,h}$ is the $\pm1$-bin frequency band around the $h$-th harmonic of note $n$, $\mathcal{T}_n$ is the set of STFT frames within 120 ms after the note onset, and $H_n \le 5$ is the number of retained harmonics below Nyquist. The narrow band handles STFT leakage while rejecting nearby pitches. The five-harmonic cap keeps PHE on lower partials that are less exposed to inharmonicity, tuning or rendering mismatch, and overlap.

**Onset-window spectral flux (OSF)** is based on spectral-flux onset functions, which are used in onset detection by capturing positive spectral differences [31, 32]. It measures the broadband, half-wave-rectified spectral increase around the note onset:

$$\mathrm{OSF}_n = \log\left(1 + \frac{1}{|\mathcal{O}_n|} \sum_{t\in\mathcal{O}_n} \sum_{k} \big[X(k,t) - X(k,t-1)\big]_+\right) \tag{9}$$

where $\mathcal{O}_n$ is the set of STFT frames from 20 ms before to 80 ms after the note onset, and $[u]_+ = \max(u,0)$ for any scalar $u$. The short pre-onset side absorbs alignment slack and provides a local reference. The longer post-onset side covers the main struck or plucked-string transient without folding in sustained energy.

### 3.2 Proxy Architecture

Figure 2 shows the proxy architecture. We use a Transformer encoder that maps a padded sequence of up to $N_{\max} = 64$ note events in a 5 s segment to the corresponding $(\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)$ pairs. We choose $N_{\max} = 64$ as our data analysis on MAESTRO reveals that human performances rarely exceed 64 notes per 5 seconds. Each note token combines a learnable pitch embedding, a shared learnable basis that maps onset, duration, and velocity (each normalized to $[0,1]$) to additive directions in token space, and a two-layer MLP with GELU activation on the normalized onset acting as a learnable positional encoding. These components are summed and passed through LayerNorm to form the note token, and then padded slots are zeroed. The encoder stacks six pre-norm Transformer layers ($d = 256$, 8 heads, FFN width $4d = 1024$, GELU, dropout 0.1), with a key-padding mask excluding padded slots from attention. A shared note-wise head ($d \to d \to 2$, with GELU) emits each $(\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)$ pair. The encoder-only design fits here because this is a set-to-set regression task.

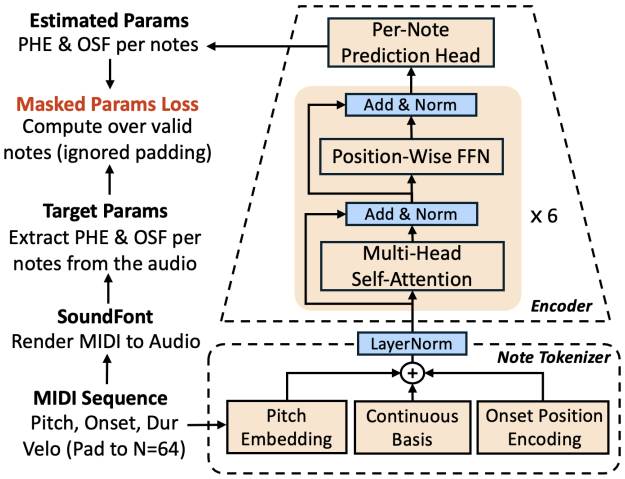



**Figure 2**. An encoder-only Transformer maps padded note sequences to per-note PHE/OSF targets extracted from SoundFont-rendered audio.

### 3.3 Training Data and Objective

Training the proxy involves three steps: MIDI segments are drawn from samplers, rendered through a SoundFont and extracted to PHE/OSF targets as teacher data, and a loss aligns the proxy's predictions with those targets.

**Note event sampling.** Note events come from a three-part mixture sampler: (1) 50% of the segments draw notes uniformly across pitch, duration, and inter-onset interval, giving broad coverage of ordinary polyphonic texture; (2)

30% are simple monophonic or low-polyphony passes that isolate how velocity alone affects a single note; (3) 20% is deliberately adversarial, with dense overlapping onsets and large chords, so that the proxy is also exposed to its worst-case inputs during training. Within any chord, per-note velocities are sampled independently so that the proxy learns per-note rather than chord-level responses.

**Teacher data from SoundFonts.** The teacher audio for the proxy is rendered using a specified SoundFont (`.sfz`) via the Python package `sfizz` at 22050 Hz. Audio is rendered in segments of 2, 5, or 10 s with reverb disabled, and short fade-outs are applied to suppress synthesizer state leakage. Rendered segments and their PHE/OSF targets are cached once and reused across epochs, decoupling proxy training from online rendering.

**Training objective.** Given the sampled note sequence $(E, V)$, the proxy predicts a $(\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)$ pair for each note. We optimize the network by minimizing the Huber loss in Eq. (3), applying a strict mask to exclude padded sequence slots from the gradient computation.

## 4. EXPERIMENTS

### 4.1 Datasets and SoundFont Selection

We use piano as the source domain, where velocity labels are available, and guitar as the main label-scarce target domain. For piano, we train on the MAESTRO V3 dataset [12] (198.7 h) using its default train/valid/test split. For guitar, we adapt on the GAPS dataset [14] (14 h) using its default train/valid/test split. Furthermore, we evaluate the cross-dataset generalization on the SMD dataset [33] (4.7 h) for piano and the François Leduc (FL) dataset [15] (4.0 h) for guitar. MAESTRO and SMD consist of Yamaha Disklavier performances. GAPS features nylon-string classical guitar, while FL comprises solo jazz guitar recordings on a mix of nylon-string, steel-string, electric, and acoustic guitars.

The SoundFont is chosen deliberately for each target instrument, since a mismatched timbre would produce a training signal that the target recordings cannot match. We audition candidates from the public FreePats sound banks and select the closest match: Salamander Grand Piano [34] for piano, which approximates the Yamaha grand piano timbre of MAESTRO and SMD, and Spanish Classical Guitar [35] for guitar, a close match for GAPS and the nylon-string performance in FL. We also include YDP Grand Piano [36], an earlier and less realistic Yamaha grand piano sample, to investigate the metric sensitivity to SoundFont change (Section 5).

### 4.2 Diff-SFProxy: Velocity-Recovery Diagnostic

**Experimental rationale.** Before deploying Diff-SFProxy in cross-instrument adaptation, we test whether its gradients can be trusted. The question is: *do the proxy's gradients point velocity in the right direction?* We answer it with a controlled inversion on synthetic audio, where the true velocities are known. This isolates the gradient pathway from any real-audio domain gap. A pass here is a precondition for Section 4.3.

#### *4.2.1 Training Protocol*

Each proxy is trained on 20k MIDI segments, with an additional 2k for validation. The optimizer is AdamW with a learning rate of $10^{-4}$, weight decay 0.01, batch size 64, and a fixed random seed 19. We train Diff-SFProxy for 200 epochs, and the validation loss typically plateaus between epochs 150 and 200. We select the checkpoint with the lowest validation loss for each run. To accommodate the three input segment lengths $2/5/10$ s, we set $N_{\max}$ to $32/64/128$ accordingly. All three proxies use the same Transformer architecture, resulting in 4.9 M parameters each.

#### *4.2.2 Diagnostic Protocol*

We render MIDI segments with the SoundFont and extract their PHE/OSF as the target. We then reset all note velocities to 64, freeze the proxy, and run gradient descent (Adam, 120 steps, learning rate 0.03) on the candidate velocities until the proxy's predicted PHE/OSF match the target. The headline metric is the per-note velocity MAE on the scale $[0, 127]$ between recovered and true velocities. A substantial reduction from the initial error indicates that the proxy's gradients yield update directions toward the true velocities, which is exactly the property needed for adaptation. We use two test groups, each containing 2k MIDI segments. The **in-domain test** draws from the same sampler used for training. The **stress test** uses denser polyphony, larger chords, and tighter inter-onset intervals to probe behavior beyond the training distribution.

#### *4.2.3 Results*

Table 1 reports the diagnostic for piano and guitar proxies at $2/5/10$ s segment lengths. In-domain, all six proxies cut the initial error to about a third. Under the stress test, recovery improves from 2 s to 5 s, most clearly on piano. From 5 s to 10 s the gain disappears, with all cells within half a MIDI unit. We therefore use the 5 s proxy as the default in the cross-instrument adaptation, and keep 2 s as an ablation. The 10 s setting adds training cost without further gain, so we discard it.

| | **In-domain** | | | **Stress** | | |
|---|---|---|---|---|---|---|
| **Diff-SFProxy** | Init | Rec. | Gain ↑ | Init | Rec. | Gain ↑ |
| Piano, 2 s | 37.2 | 12.3 | 66.9% | 38.6 | 19.5 | 49.5% |
| **Piano, 5 s** | 37.9 | 12.2 | **67.8%** | 38.5 | 17.7 | **54.0%** |
| Piano, 10 s | 38.2 | 12.5 | 67.3% | 38.5 | 17.8 | 53.7% |
| Guitar, 2 s | 33.6 | 9.3 | 72.3% | 34.2 | 11.3 | 67.0% |
| **Guitar, 5 s** | 34.6 | 9.1 | **73.7%** | 33.8 | 10.7 | **68.3%** |
| Guitar, 10 s | 34.6 | 9.5 | 72.5% | 33.8 | 10.8 | 68.0% |

**Table 1**. Velocity recovery diagnostic. Init and Rec. report per-note velocity MAE in MIDI units $[0, 127]$ before and after optimization; Gain $=$ (Init $-$ Rec.)/Init $\times$ 100%.

| Method | Piano | | | | | | Guitar | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | MAESTRO test set | | | SMD dataset | | | GAPS test set | | FL dataset | |
| | $\mathrm{MAE}_{\mathrm{Velo}} \downarrow$ | $r_{\mathrm{BSSL}} \uparrow$ | $r_{\mathrm{BSTL}} \uparrow$ | $\mathrm{MAE}_{\mathrm{Velo}} \downarrow$ | $r_{\mathrm{BSSL}} \uparrow$ | $r_{\mathrm{BSTL}} \uparrow$ | $r_{\mathrm{BSSL}} \uparrow$ | $r_{\mathrm{BSTL}} \uparrow$ | $r_{\mathrm{BSSL}} \uparrow$ | $r_{\mathrm{BSTL}} \uparrow$ |
| **Flat Velocity:** set at 64 | 14.8 | 0.757 | 0.622 | 15.4 | 0.701 | 0.582 | 0.737 | 0.701 | 0.714 | 0.587 |
| **VeloEst:** pretrained on MAESTRO | **3.9** | **0.878** | **0.892** | **8.0** | **0.829** | **0.816** | 0.769 | 0.798 | 0.740 | 0.699 |
| **VeloEst + Diff-Synth** | 19.2 | 0.776 | 0.752 | 19.7 | 0.789 | 0.742 | 0.669 | 0.624 | 0.646 | 0.547 |
| - uses 2s backend instead of 5s | 25.0 | 0.702 | 0.553 | 25.9 | 0.662 | 0.520 | 0.760 | 0.770 | 0.729 | 0.636 |
| **VeloEst + Diff-SFProxy** (proposed) | <u>10.5</u> | <u>0.869</u> | <u>0.881</u> | <u>11.3</u> | <u>0.821</u> | <u>0.806</u> | **0.794** | **0.860** | **0.777** | **0.788** |
| - uses 2s backend instead of 5s | 10.9 | 0.858 | 0.870 | 11.8 | 0.820 | 0.804 | <u>0.787</u> | <u>0.836</u> | <u>0.771</u> | <u>0.756</u> |
| - w/o anti-collapse and saturation loss | 12.9 | 0.789 | 0.698 | 14.4 | 0.735 | 0.653 | 0.740 | 0.705 | 0.723 | 0.614 |
| - w/o VeloEst pretrained weights | 20.5 | 0.770 | 0.757 | 21.2 | 0.723 | 0.703 | 0.702 | 0.686 | 0.683 | 0.600 |

**Table 2**. Evaluation on piano and guitar datasets. Diff-Synth and Diff-SFProxy backends process 5 s segments by default. Best and second-best results are marked in bold and underlined, respectively.

### 4.3 Cross-Instrument VeloEst Adaptation

**Experimental rationale.** Our experiment on adaptation has two purposes. On guitar, where velocity labels are unavailable, it provides a convenient label-free approach to MIDI velocity estimation. On piano, where ground-truth velocity is available on MAESTRO and SMD, it serves as a controlled diagnostic: we adapt on MAESTRO so that predictions can be checked against true labels, verifying whether our non-velocity supervision pushes velocity toward the right value. The piano result is therefore not a claim about improving over a supervised piano baseline. It is a sanity check that the supervision signal is informative.

#### *4.3.1 Training Protocol*

We compare three settings on the same VeloEst front-end: the piano-pretrained baseline (no adaptation), Diff-Synth adaptation, and Diff-SFProxy adaptation. The Diff-Synth backends [21, 22] were originally implemented for 3 s input segments. We retrain each at 2 s and 5 s with their original hyperparameters, then freeze them.

During adaptation, VeloEst processes 10 s segments, while the frozen backend operates on 5 s crops by default and 2 s for ablation. Backend losses are averaged over valid notes in each crop and backpropagated to the corresponding VeloEst 10 s predictions, keeping the gradient scale comparable across backend crop lengths. Adaptation is run separately on the MAESTRO and GAPS training sets for 120k iterations using Adam with learning rate $10^{-4}$, batch size 8, and step decay 0.9 every 10k iterations. SMD and FL are used only for evaluation. All runs use seed 19, and the final checkpoint is selected by validation $r_{\mathrm{BSSL}}$ on MAESTRO for piano or GAPS for guitar.

#### *4.3.2 Evaluation Metrics*

For piano datasets with ground-truth velocities, we follow [10, 11] and report per-note $\mathrm{MAE}_{\mathrm{Velo}}$. For datasets without velocity labels, $\mathrm{MAE}_{\mathrm{Velo}}$ cannot be computed. We instead re-render the aligned notes through the selected SoundFont using the predicted velocities, and compare the resynthesis with the original recording in perceptual loudness space.

We use Bark-scale specific loudness (BSSL) and Bark-scale total loudness (BSTL) [26, 27] as perceptually grounded comparison features. BSSL is a frame-wise critical-band representation, while BSTL is its band-summed contour, tracking perceived loudness over time. Both features capture the main perceptual effect targeted by velocity estimation, namely the temporal shaping of loudness, while reducing sensitivity to timbral and recording-condition mismatch between real audio and SoundFont resynthesis. We avoid two natural alternatives. Raw RMS energy ignores frequency-dependent perceptual sensitivity. Note-wise loudness measures, including PHE and OSF, are onset-locked, miss the continuous loudness contour, and coincide with the proxy training targets, making the evaluation circular.

Pearson correlation ($r$) between real and rendered loudness contours is our primary metric. Because it is invariant to global scale and offset, it reduces sensitivity to loudness differences introduced by gain, microphone response, or renderer calibration. We denote Pearson correlation computed on BSSL and BSTL as $r_{\mathrm{BSSL}}$ and $r_{\mathrm{BSTL}}$, respectively. Cosine similarity ($cs$) and MAE are retained as diagnostic alternatives: cosine similarity is scale-invariant but not offset-invariant, whereas MAE is sensitive to both. Their behaviour is examined in Section 5.

#### *4.3.3 Results*

Table 2 reports evaluation across all four test sets. The row labeled 'VeloEst: pretrained on MAESTRO' is the supervised checkpoint applied without further training, serving as the zero-shot transfer baseline for SMD, GAPS, and FL. On piano, this checkpoint remains the strongest supervised reference. Both adaptation strategies underperform on piano because they optimize indirect performance-matching objectives rather than the available velocity labels.

On guitar, Diff-SFProxy achieves the best $r_{\mathrm{BSSL}}$ and $r_{\mathrm{BSTL}}$ on both GAPS and FL, outperforming flat velocity, zero-shot transfer, and Diff-Synth under both backend crop lengths. Diff-Synth does not reliably improve guitar performance and can degrade it: on FL, the 5 s Diff-Synth backend falls below flat velocity in $r_{\mathrm{BSSL}}$ (0.646

vs. $0.714$). This supports the concern in Section 2.2: waveform-level objectives are more exposed to timbral and room-acoustic mismatch than parameter-level supervision. The crop-length effect also differs sharply between the two approaches. On FL, increasing the backend crop from $2\,\mathrm{s}$ to $5\,\mathrm{s}$ reduces Diff-Synth's $r_{\mathrm{BSSL}}$ by $0.083$, whereas Diff-SFProxy changes by only $0.006$. Longer crops expose waveform-level objectives to more residual mismatch, while parameter-level supervision remains focused on note-wise velocity-related cues. The zero-shot piano checkpoint already exceeds flat velocity on both guitar sets, suggesting partial cross-instrument transfer, while Diff-SFProxy provides a consistent further improvement.

#### *4.3.4 Ablations*

The bottom rows of Table 2 isolate two design choices. Removing $\mathcal{L}_{\mathrm{anti}}+\mathcal{L}_{\mathrm{satu}}$ gives a plausible $\mathrm{MAE}_{\mathrm{Velo}}$ on MAESTRO, but $r_{\mathrm{BSSL}}$ falls near the flat-velocity baseline. The outputs collapse toward a near-constant value, consistent with the median-seeking behavior of MAE: central predictions can keep absolute error tolerable while losing the dynamic contour. Removing the piano-pretrained checkpoint leaves the proxy loss without a supervised velocity anchor, causing performance to drift across all datasets and underperform flat velocity on three of the four guitar metrics. Piano pretraining therefore acts as a crucial prior for cross-instrument adaptation.

## 5. DISCUSSION

**Scope and metric implications.** Velocity is not the only factor that shapes rendered audio loudness: pedaling, continuous controllers, and effects such as reverb also contribute [37]. Diff-Synth backends such as DDSP-Piano expose several of these controls. Consequently, when adaptation updates velocity alone while the remaining controls are fixed or unobserved, the velocity gradient can be confounded by residuals attributable to those controls. Diff-SFProxy instead restricts the optimization to velocity, making adaptation more tractable. This simplification comes at a cost: the recovered velocity may absorb effects that, in the real performance, arose from other controls. Hence, the recovered velocity is not a canonical property of the acoustic performance. It is defined relative to the chosen SoundFont as the velocity that best reproduces the perceptual dynamics of the original audio. This renderer-conditioned velocity is therefore the relevant target for label-scarce expressive analysis or resynthesis. Accordingly, the evaluation metric must be robust to SoundFont choice and sensitive to dynamics differences.

**Why Pearson $r$ on BSSL/BSTL.** Table 3 tests metrics against the two requirements implied above: sensitivity to expressive dynamics and robustness to renderer-specific loudness calibration. To test dynamics sensitivity, $\Delta(\mathrm{GT}-\mathrm{Flat}\,64)$ asks whether a metric can tell expressive resynthesis from a flat one. Among the similarity metrics, Pearson $r$ gives the larger separation on both BSSL ($+0.118$) and BSTL ($+0.263$). Cosine similarity also orders GT above flat, but severely compresses the margin ($+0.052$ on BSSL and $+0.047$ on BSTL), proving **insensitive** to fine dynamics differences. To evaluate renderer robustness, $\Delta|\mathrm{Sal.}-\mathrm{YDP}|$ asks whether a metric is stable when the rendering SoundFont is swapped. Here, MAE proves **over-sensitive** to absolute gain: it drifts by $0.193$ on BSSL and $1.723$ on BSTL, substantially more than the corresponding drift on $r$ ($0.034$, $0.011$). On real recordings, this drift would conflate dynamics differences with renderer choice. Overall, Pearson $r$ on BSSL/BSTL provides the ideal trade-off, whereas cosine similarity and MAE fail on sensitivity and robustness, respectively, and are kept only as diagnostic alternatives.

| Method | MAESTRO test set | | | | | |
|---|---|---|---|---|---|---|
| | BSSL | | | BSTL | | |
| | $r$ | $cs$ | MAE | $r$ | $cs$ | MAE |
| **GT velocity** | 0.870 | 0.939 | 0.702 | 0.883 | 0.975 | 3.706 |
| - uses YDP Piano | 0.836 | 0.926 | 0.510 | 0.872 | 0.973 | 1.983 |
| $\Delta\lvert\mathrm{Sal.}-\mathrm{YDP}\rvert\downarrow$ | <u>0.034</u> | **0.013** | 0.193 | <u>0.011</u> | **0.002** | 1.723 |
| **Flat velocity = 64** | 0.752 | 0.887 | 0.786 | 0.620 | 0.928 | 4.065 |
| - uses YDP Piano | 0.733 | 0.883 | 0.637 | 0.617 | 0.930 | 2.750 |
| $\Delta\lvert\mathrm{Sal.}-\mathrm{YDP}\rvert\downarrow$ | <u>0.019</u> | **0.004** | 0.149 | <u>0.003</u> | **0.002** | 1.314 |
| $\Delta$ **(GT − Flat)** $\uparrow$ | **+0.118** | <u>+0.052</u> | -0.083 | **+0.263** | <u>+0.047</u> | -0.359 |

**Table 3**. MAESTRO metric diagnostic. Real audio is compared with GT-note resynthesis using GT or flat velocity under the Salamander (default) or YDP Grand Piano. Summary rows show GT–Flat separation and SoundFont drift.

## 6. CONCLUSION

In this paper, we presented a framework for cross-instrument MIDI velocity estimation. A piano-pretrained estimator is adapted to target instruments without velocity labels through one of two differentiable interfaces: waveform-space Diff-Synth or loudness-related acoustic parameter-space Diff-SFProxy. The proposed Diff-SFProxy leverages note-wise loudness parameters extracted from black-box SoundFont renders, focusing the gradient on velocity-dependent behavior rather than waveform details. On guitar, where velocity labels are unavailable, Diff-SFProxy improves over zero-shot transfer and waveform-level adaptation, while Diff-Synth often degrades performance. Our finding suggests that, when the task is to estimate a single control parameter rather than reconstruct audio, parameter-wise supervision through a renderer proxy is more reliable than waveform supervision.

Future work includes extending the framework to other instruments, such as violin and wind instruments. Automating SoundFont selection through timbre matching would replace the current by-ear procedure and scale the method to broader datasets. Beyond velocity, the framework could also recover other expressive control parameters, such as sustain pedaling in piano and bow-pressure curves in violin.

## 7. ACKNOWLEDGMENTS

This research was carried out while the author was in receipt of a University International Fees Scholarship and a University Postgraduate Award at The University of Western Australia. The authors acknowledge the use of Kaya, the High-Performance Computing facility at The University of Western Australia, and the technical assistance provided by the UWA IT – HPC team.

## 8. REFERENCES

[1] R. B. Dannenberg, "The interpretation of midi velocity," in *Proceedings of the International Computer Music Conference (ICMC)*, 2006, pp. 193–196.

[2] D. Jeong, T. Kwon, and J. Nam, "A timbre-based approach to estimate key velocity from polyphonic piano recordings," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 120–127.

[3] S. Rhyu, S. Kim, and K. Lee, "Sketching the expression: Flexible rendering of expressive piano performance with self-supervised learning," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 178–185.

[4] H. Zhang, S. Chowdhury, C. E. Cancino-Chacón, J. Liang, S. Dixon, and G. Widmer, "Dexter: Learning and controlling performance expression with diffusion models," *Applied Sciences*, vol. 14, no. 15, p. 6543, 2024.

[5] J. Tang, E. Cooper, X. Wang, J. Yamagishi, and G. Fazekas, "Towards an integrated approach for expressive piano performance synthesis from music scores," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[6] I. Borovik, D. Gavrilev, and V. Viro, "SyMuPe: Affective and controllable symbolic music performance," in *Proceedings of the 33rd ACM International Conference on Multimedia*, ser. MM '25. Dublin, Ireland: Association for Computing Machinery, 2025, pp. 10 699–10 708.

[7] A. Morsi, S. Chiruthapudi, S. Peter, I. Pilkov, L. Bishop, A. Maezawa, X. Serra, and C. E. Cancino-Chacón, "Enabling empirical analysis of piano performance rehearsal with the rach3 midi dataset," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2025, pp. 484–491.

[8] H. Zhang, J. Tang, S. R. M. Rafee, S. Dixon, and G. Fazekas, "ATEPP: A dataset of automatically transcribed expressive piano performance," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 446–453.

[9] F. Simonetta, S. Ntalampiras, and F. Avanzini, "Acoustics-specific piano velocity estimation," in *Proceedings of the IEEE International Workshop on Multimedia Signal Processing (MMSP)*, 2022.

[10] H. Kim and X. Serra, "A method for midi velocity estimation for piano performance by a U-net with attention and FiLM," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 304–310.

[11] Z. He, R. Togneri, and D. Huang, "Score-informed transformer for refining midi velocity in automatic music transcription," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2026.

[12] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the maestro dataset," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[13] N. C. Tamer, Y. Özer, M. Müller, and X. Serra, "High-resolution violin transcription using weak labels," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 223–230.

[14] X. Riley, Z. Guo, A. C. Edwards, and S. Dixon, "GAPS: a large and diverse classical guitar dataset and benchmark transcription model," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 611–617.

[15] X. Riley, D. Edwards, and S. Dixon, "High resolution guitar transcription via domain adaptation," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1051–1055.

[16] B. Maman and A. H. Bermano, "Unaligned supervision for automatic music transcription in the wild," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2022, pp. 14 918–14 934.

[17] Y. Zang, Y. Zhong, F. Cwitkowitz, and Z. Duan, "SynthTab: Leveraging synthesized data for guitar tablature transcription," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1286–1290.

[18] Y. Kusaka and A. Maezawa, "Learn from virtual guitar: A comparative analysis of automatic guitar transcription using synthetic and real audio," in *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2025, pp. 1–5.

[19] G. Sato and T. Akama, "Annotation-free automatic music transcription with scalable synthetic data and adversarial domain confusion," in *Proceedings of the 2024 IEEE International Conference on Multimedia and Expo (ICME)*, 2024, pp. 1–6.

[20] T.-K. Wang, Y.-P. Peng, L. Su, and V. K. M. Cheung, "Vioptt: Violin technique-aware transcription from synthetic data augmentation," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 15 947–15 951.

[21] L. Renault, R. Mignot, and A. Roebel, "DDSP-Piano: A neural sound synthesizer informed by instrument knowledge," *Journal of the Audio Engineering Society*, vol. 71, no. 9, pp. 552–565, 2023.

[22] N. Jonason, X. Wang, E. Cooper, L. Juvela, B. L. T. Sturm, and J. Yamagishi, "DDSP-Based neural waveform synthesis of polyphonic guitar performance from string-wise midi input," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2024, pp. 208–215.

[23] M. A. Martínez Ramírez, O. Wang, P. Smaragdis, and N. J. Bryan, "Differentiable signal processing with black-box audio effects," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021.

[24] O. Barkan, S. Shvartzman, N. Uzrad, M. Laufer, A. Elharar, and N. Koenigstein, "InverSynth II: Sound matching via self-supervised synthesizer-proxy and inference-time finetuning," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 642–648.

[25] P. Combes, S. Weinzierl, and K. Obermayer, "Neural proxies for sound synthesizers: Learning perceptually informed preset representations," *Journal of the Audio Engineering Society*, vol. 73, no. 9, pp. 561–577, 2025.

[26] E. Zwicker and H. Fastl, *Psychoacoustics: Facts and Models*, 2nd ed., ser. Springer Series in Information Sciences. Berlin: Springer, 1999, vol. 22.

[27] Z. He, H. Meng, D. Huang, and R. Togneri, "Joint estimation of piano dynamics and metrical structure with a multi-task multi-scale network," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 14 607–14 611.

[28] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "DDSP: differentiable digital signal processing," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[29] Y. Wu, E. Manilow, Y. Deng, R. Swavely, K. Kastner, T. Cooijmans, A. Courville, C.-Z. A. Huang, and J. Engel, "MIDI-DDSP: Detailed control of musical performance via hierarchical modeling," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[30] S. Ewert and M. Müller, "Estimating note intensities in music recordings," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011, pp. 385–388.

[31] S. Dixon, "Onset detection revisited," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2006, pp. 133–137.

[32] S. Böck and G. Widmer, "Maximum filter vibrato suppression for onset detection," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2013, pp. 55–61.

[33] M. Müller, V. Konz, W. Bogler, and V. Arifi-Müller, "Saarland music data (smd)," in *Late-Breaking and Demo Session of the 12th Int. Conf. on Music Information Retrieval*, 2011.

[34] A. Holm, "Acoustic grand piano: Salamander Grand Piano," FreePats Project sound bank, Jun. 2020, version V3+2020-06-02; Creative Commons Attribution 3.0; accessed 2026-04-18. [Online]. Available: https://freepats.zenvoid.org/Piano/acoustic-grand-piano.html

[35] FreePats Project, "Nylon-string acoustic guitar: Spanish Classical Guitar," FreePats Project sound bank, Jun. 2019, version 2019-06-18; Creative Commons CC0 1.0 public domain dedication; accessed 2026-04-18. [Online]. Available: https://freepats.zenvoid.org/Guitar/acoustic-guitar.html

[36] ——, "Acoustic grand piano: YDP Grand Piano," FreePats Project sound bank, Aug. 2016, version 2016-08-04; Creative Commons Attribution 3.0; accessed 2026-04-18. [Online]. Available: https://freepats.zenvoid.org/Piano/acoustic-grand-piano.html

[37] J. Ryu, S. Rhyu, H.-G. Yoon, E. Kim, J. Y. Yang, and T. Kim, "Mid-fild: Midi dataset for fine-level dynamics," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024, pp. 222–230.